Operating System Project - Phase 2

Memory Management with Paging and NRU Page Replacement

Introduction

Memory management is a fundamental responsibility of any modern operating system. As processes compete for limited physical memory, the OS must provide mechanisms that ensure efficiency, fairness, isolation, and high overall performance. Paging is one of the most widely used memory management techniques, offering a practical solution for eliminating external fragmentation while simplifying memory allocation.

In this project, you will explore how paging works. You will model virtual and physical address spaces, design and manage page tables, simulate address translation, handle page faults, and evaluate system behaviour under demand paging. The primary page replacement policy in this phase is NRU (Not Recently Used).

Objectives

- Understand paging fundamentals and the role of demand paging in memory management.
- Implement the core data structures required for physical memory, page tables, and disk-backed pages.
- Design and integrate a page replacement mechanism using the NRU (Not Recently Used) algorithm.
- Handle page faults correctly, including blocking, swapping, and page-table updates.
- Develop debugging and verification skills on Linux.

System Description & Assumptions

- The system has a single CPU with a 10-bit byte-addressable memory, meaning each address refers to one byte. This phase extends the 1-CPU Round Robin implementation from Phase 1 only; students do not need to adapt HPF for this phase.
- Physical memory (RAM) size is 512 bytes.
- Page size is 16 bytes.
- Therefore, RAM contains 32 physical page frames.
- In this phase, the terms cycle, tick, and second all refer to the same integer unit of the project clock. A memory access takes 1 tick (1 second), while a disk access takes 10 ticks (10 seconds).
- NRU (Not Recently Used) is used as the primary page replacement algorithm.
- Pages required by the OS itself are omitted from the simulation for simplicity.
- Each resident data page must track at least two status bits: Referenced (R) and Modified (M).
- To make NRU meaningful, the OS should periodically clear the R bits of resident data pages. For this project, the R-bit clearance timeout is a fixed number of Round Robin quantums, denoted by K. The simulator should prompt the user at startup for K, and whenever K quantums have elapsed, all R bits of resident data pages are cleared.
- Frames containing process page tables are not candidates for replacement and must remain resident once allocated.

Requirements

You are required to edit the scheduler (scheduler.c) from the OS Scheduler project and add an MMU module (for example, MMU.c) to provide paging and memory-allocation capabilities. The MMU should allocate pages for processes as they request them, swap pages when needed, and free frames when processes finish so that memory can be reused by later processes. Only Round Robin (RR) is required in this phase. In addition to the

RR quantum, the simulator should ask the user for the fixed number of quantums K used as the timeout for clearing all R bits of resident data pages.

1. Define a structure for physical frames so the system can determine which frames are free and which are occupied. You may use a free linked list, bitmap, array, or any equivalent structure.
2. Create the data structures needed to perform page replacement using NRU. For every occupied data frame, you should be able to determine: the owning process, the loaded virtual page, whether the frame was recently referenced, and whether it was modified.
3. Decide the page-table entry format and size. At minimum, each entry should store the physical frame number when the page is resident, plus any flags you need such as valid/present, referenced, and modified. (Hint: you do not store a virtual frame number inside the entry.)
4. Whenever the OS needs a frame, it must first search for a free frame before attempting replacement.
5. When a process starts, the OS must allocate one free frame for that process page table. Initially, all page-table entries are invalid.
6. If no free frame exists when a page table must be allocated, the OS should use NRU to evict a resident data page and assign the freed frame to the new page table. Page-table frames themselves must never be chosen as victims.
7. The PCB must be updated with the physical address (or frame number) of the process page table. Each process has its own page table.
8. Load the first page of the process into memory when the process starts, then update the corresponding page-table entry.
9. For simplicity, assume that only the first page of the page table is needed and that initial page-table allocation plus initial first-page loading take no extra time.
10. Do not swap out process page tables.
11. A page-table frame remains resident for the lifetime of its process. NRU victim selection must consider resident data/code pages only; page-table frames are excluded from all replacement classes.
12. When a page fault occurs, the OS must demand the required page, update the corresponding page table, and block the process during disk activity. The blocked process leaves the CPU, and the scheduler may dispatch another ready process.
13. The 1-tick context-switch overhead defined in Phase 1 still applies whenever the CPU switches from one process to another, including when a running process faults and the scheduler dispatches a different ready process. The disk transfer itself proceeds while the faulting process is blocked.
14. The requested page is referenced by the process at specific moments during its runtime according to the request file.
15. If no free frame exists during a page fault, the OS must apply NRU over resident data pages only. The victim-selection rule is as follows: classify pages into the four NRU classes (0: R=0,M=0), (1: R=0,M=1), (2: R=1,M=0), and (3: R=1,M=1), then choose a victim from the lowest non-empty class. For deterministic behaviour, scan physical frames in ascending order and evict the first frame found in the selected class.
16. If the chosen victim page is modified, it must be written back to disk before replacement. In that case, the disk is accessed twice: once to write the victim and once to load the demanded page.
17. Initially assume that all memory frames are free.
18. Any access to a resident page should set its Referenced bit. Any write access should also set its Modified bit.

Input

You will need to slightly modify the Process Generator (process_generator.c) to accept extra process information: the base and limit of the process on disk. Here, base is the starting page number of the process image on disk, and limit is the number of virtual pages that belong to that process.

processes.txt example

```
#id     arrival  runtime priority base    limit

1       1        6       5        0       4

2       3        3       3        20      3
```

- Comments are lines beginning with # and should be ignored.
- Each non-comment line represents one process.
- Fields are separated with one tab character ('\t').
- You can assume that processes are sorted by arrival time.
- Two or more processes may arrive at the same time.

You will also have one request file per process representing the virtual addresses requested by that process. The time in the request file is relative to the CPU time consumed by that process since it first started, not absolute wall-clock time. For example, if a process first starts at time 8 and later accumulates two total CPU ticks, then a request at time 2 is triggered at that moment.

requests.txt example

```
#time  addressInBinary      r/w

1      0       r

2      10000   r

3      100000 w
```

Output

A new output file named memory.log should be generated for memory-related events. No scheduler.log output is required in this phase; memory.log is the only required grading log.

memory.log example

```
# PageFault upon VA "xx" from process "yy"
# Free Physical page "ZZ" allocated
# Swapping out page "ZZ" to disk
# At time "i" disk address "X" for process "y" is loaded into memory page "ZZ".


## in case of no free page and no write-back is needed

PageFault upon VA 100110 from process 1
At time 1 disk address 2 for process 1 is loaded into memory page 10.



## in case of free page
PageFault upon VA 10 from process 2
Free Physical page 2 allocated
At time 1 disk address 20 for process 2 is loaded into memory page 2.
```

- Comments and empty lines are ignored during automatic checking.
- You must follow the required format exactly because output files are compared automatically.
- You may add extra internal debug prints to the console, but the required log file must match the expected format.

NRU Notes

- NRU does not require a queue structure like Second Chance. Instead, it depends on maintaining correct R and M bits for resident data pages.
- The periodic reset of R bits is essential; otherwise, all pages may eventually look recently used forever. In this project, the reset happens after every K quantums, where K is a user-specified fixed timeout entered at startup.
- Because modified pages are more expensive to replace, NRU naturally prefers clean, not recently used pages whenever possible.
- Page-table frames must be excluded from NRU classification and victim selection.

Guidelines

- Read the document carefully at least once before implementation.
- Your program must not crash.
- Spend time on the design first; it will make implementation much easier.
- The code should be clearly commented, and variable names should be meaningful.
- Correct any mistakes discussed during Phase 1.
- Violation of academic integrity leads to a negative project grade.

Deliverables

You should deliver a public repository containing the full project along with a final report. The report should include all relevant system details, such as the block diagram, process states, memory-management design, page-table structure, NRU victim-selection logic, and any additional assumptions you made.

Grading

- Correct data structures, page-table structure, and memory-management logic (20%).
- Correct swapping and page-fault handling (10%).
- Correct process blocking during disk activity and proper return to the ready queue after faults (10%).
- Correct integration with Round Robin scheduling, including context-switch handling after faults (20%).
- Correct NRU implementation (20%).
- Correct integration and output format (20%).
- Plagiarism (-100%).